

Programación III - Universidad Nacional de General Sarmiento

Ejercicios: Complejidad computacional

1. ¿Qué es la complejidad computacional de un algoritmo? ¿Por qué se mide el peor caso?
2. ¿Cuándo decimos que un algoritmo es polinomial?
3. ¿Cuándo decimos que un algoritmo es exponencial? ¿Qué algoritmos de complejidad exponencial conoce?
4. ¿Cuándo decimos que un problema está bien resuelto?
5. Si tenemos un algoritmo con complejidad $O(n \log n)$ y otro algoritmo con complejidad $O(n)$, cuál es preferible en la práctica? ¿Existe alguna circunstancia que cambie la respuesta a la pregunta anterior?

6. Calcular la complejidad computacional del siguiente algoritmo (y comentar qué hace el algoritmo!):

```
public Map<Integer , Integer> cantidades (ArrayList<Integer> A)
{
    Map<Integer , Integer> ret = new TreeMap<Integer , Integer>();

    for (Integer elem: A)
        ret.put(elem, apariciones(A, elem));

    return ret;
}

private int apariciones (ArrayList<Integer> A, Integer elem)
{
    int ret = 0;
    for (Integer valor: A)
        ret += elem == valor ? 1 : 0;

    return ret;
}
```

7. ¿Se puede dar un algoritmo de menor complejidad para el problema del ejercicio anterior? ¿Cómo depende la complejidad de la implementación particular de Map que se seleccione?

8. Calcular la complejidad computacional del siguiente algoritmo, sabiendo que el método $proc(x)$ tiene complejidad $O(n)$:

```
public void poochie( ArrayList<Integer> A)
{
    int n = A.size() - 1;
    int ret = 0;

    while( n > 0 )
    {
        ret += proc( A.get(n) );
        n = n/2;
    }

    return ret;
}
```

9. ¿Qué es un algoritmo de tipo *divide and conquer*? ¿Qué algoritmos de este tipo conoce?
¿Qué se puede decir de la complejidad computacional de estos algoritmos?
10. ¿Cuál es la complejidad computacional de la búsqueda binaria? ¿Se puede buscar un elemento en un arreglo no ordenado en tiempo menor que $O(n)$?